UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Large scale data management

Conversione di PuckTrick a SPARK

Autori:
Daniel Satriano - m.919053
Francesco Cavallini - m.920835

Anno accademico:
2025/2026

Indice

1 Introduzione **1**
1.1 Obbiettivo del progetto 1
1.1.1 Cos'è SPARK 1
1.1.2 Decisioni progettuali 2
1.2 Esempio di utilizzo di PuckTrick 3

2 Sfide Tecniche **5**
2.1 Assenza di indici posizionali in Spark 5
2.2 Lazy evaluation vs eager evaluation 6
2.3 Campionamento DataFrame distribuiti 6
2.4 Gestione dei valori nulli 7
2.5 Gestione dei path 7
2.6 Attendibilità dei test 8

3 Spark Setup **9**
3.1 Configurazione accesso SSH tra i nodi 9
3.2 Registrazione dei worker e configurazione dell'ambiente 10
3.2.1 Criticità di questa fase 11
3.3 Creazione delle directory di logging 11
3.4 Apertura delle porte di rete. 11
3.5 Verifica del funzionamento 12

4 Performance analisys **13**
4.1 Analisi dei risultati di performance: confronto Pandas vs Spark 13
4.1.1 Protocollo sperimentale 13
4.1.2 Disclaimer: 14
4.1.3 Risultati del Paragone tra Backends 15
4.2 Spark stress test 18
4.2.1 Strategy di Test Utilizzata 18

Bibliografia **21**

Abstract

PuckTrick è una libreria Python open-source, sviluppata presso l'Università degli Studi di Milano-Bicocca, progettata per introdurre errori controllati all'interno di dataset tabulari destinati al machine learning. La libreria consente di simulare cinque tipologie di errore nei dati: duplicati, etichette inconsistenti, valori mancanti, rumore e outlier, ciascuna configurabile tramite una strategia dichiarativa in formato dizionario.

Il presente lavoro, svolto nell'ambito del branch Spark_Conversion, ha avuto come obiettivo principale l'estensione della libreria PuckTrick per supportare Apache Spark (PySpark) come motore di elaborazione alternativo a Pandas. L'intento è permettere alla libreria di operare su dataset di grandi dimensioni in modo distribuito, mantenendo la stessa interfaccia utente e garantendo risultati funzionalmente equivalenti tra i due backend.

Capitolo 1

Introduzione

1.1 Obbiettivo del progetto

Nell'ambito del machine learning, la qualità dei dati è un fattore determinante per l'affidabilità dei modelli. Tuttavia, nel mondo reale i dataset sono spesso affetti da errori di varia natura (valori mancanti, duplicati, rumore, etichette errate, outlier) che possono compromettere le prestazioni dei sistemi predittivi. PuckTrick nasce con lo scopo di fornire uno strumento riproducibile e configurabile per **simulare questi errori in modo controllato**, permettendo a ricercatori e professionisti di:

- **Valutare la robustezza** dei propri modelli di machine learning rispetto a dati corrotti
- **Generare benchmark realistici** per il confronto di tecniche di data cleaning e data quality
- **Studiare l'impatto** di specifiche tipologie di errore sulle performance predittive

La libreria nella sua origine si propone quindi come strumento di supporto alla ricerca nel campo della **Data Quality** e del **Robust Machine Learning**. Ampliando poi lo sviluppo della libreria nella sua versione spark ne si aumentà la flessibilità e si abilita la possibilità di ricerca in campi di **Data Quality** anche per **Large Models**.

1.1.1 Cos'è SPARK

Apache Spark è un framework open-source per l'elaborazione distribuita di dati su larga scala, originariamente sviluppato presso l'UC Berkeley e oggi mantenuto dalla Apache Software Foundation [**<empty citation>**]. A differenza di Pandas, che opera in memoria su una singola macchina, Spark distribuisce il carico computazionale su un **cluster di nodi**, consentendo di processare dataset dell'ordine di terabyte.

Le caratteristiche chiave di Spark rilevanti per questo progetto sono:

- **Elaborazione lazy**: le trasformazioni vengono pianificate ma eseguite solo quando si richiede un risultato concreto (azione), permettendo ottimizzazioni automatiche del piano di esecuzione

- **API DataFrame**: un'interfaccia tabulare simile concettualmente a Pandas, ma basata su operazioni colonnari distribuite
- **PySpark**: il binding Python di Spark, che consente di utilizzare l'ecosistema Spark direttamente da codice Python
- **Scalabilità orizzontale**: la possibilità di aggiungere nodi al cluster per gestire volumi di dati crescenti

L'integrazione di Spark in PuckTrick consente alla libreria di superare i limiti di memoria e performance imposti da Pandas quando si lavora con dataset di grandi dimensioni. Avendo che la differenza che la differenza tra pandas e spark è che, mentre Pandas a livello di righe, Spark lavora a livello di colonne; otimizzando le operazioni di accesso ai dati.

1.1.2 Decisioni progettuali

Abbiamo deciso di intraprendere un percorso diverso rispetto la struttura progetturale originaria di PuckTrick, guidata dalla necessità di implementare in parallelo anche la versione Spark dei metodi pandas. Per rispondere alle necessità abbiamo creato una classe PuckTrick che viene esposta all'utente che utilizza la libreria. In particolare gli permetterà di creare un oggetto con il quale potrà richiamare dinamicamente i metodi per l'introduzione del rumore all'interno dei dataset.

1. **Ristrutturazione del progetto**: riorganizzazione della struttura del repository secondo un layout modulare

   (a) `pucktrick/backends/pandas_backend/`

   (b) `pucktrick/backends/spark_backend/`

   (c) `pucktrick/backends/core/`

   con introduzione di un sistema di logging centralizzato e utility condivise.

2. **Architettura multi-backend**: progettazione e implementazione di un'architettura a plugin basata sul pattern *Strategy/Factory*, con un sistema di selezione automatica del backend (`Engine.AUTO`, `Engine.PANDAS`, `Engine.SPARK`) in funzione della disponibilità di PySpark e delle dimensioni del dataset.

3. **Conversione completa dei moduli**: riscrittura in PySpark di tutti e cinque i moduli di iniezione errori (`duplicated`, `labels`, `missing`, `noisy`, `outliers`) e delle relative utility, rispettando il paradigma colonnare e lazy di Spark ed evitando operazioni row-by-row.

4. **Supporto cluster remoto**: implementazione della possibilità di connettersi a cluster Spark remoti tramite un dizionario di configurazione, con un metodo statico dedicato (`make_remote_cluster_config`).

5. **Test e validazione**: sviluppo di una suite di **unit test specifici per Spark** e di **test di integrazione cross-backend** che verificano la coerenza funzionale tra le uscite Pandas e Spark.

1.2 Esempio di utilizzo di PuckTrick

Tramite la ristrutturazione del progetto è ora possibile accedere ai servizi della libreria tramite la seguente struttura logica:

```
Utente
  |
  v
PuckTrick(df, engine=Engine.AUTO)
  |
  v
BackendFactory.get_backend()
  |
  |-- Engine.PANDAS --> PandasBackend
  |       |-- duplicated.py
  |       |-- labels.py
  |       |-- missing.py
  |       |-- noisy.py
  |       |-- outliers.py
  |       `-- utils.py
  |
  `-- Engine.SPARK --> SparkBackend
          |-- duplicated.py
          |-- labels.py
          |-- missing.py
          |-- noisy.py
          |-- outliers.py
          |-- utils_spark.py
          `-- spark_session.py
```

Di seguito, un esempio di inizializzazione della libreria tramite codice:

```
import pucktrick
//...

strategy = {
    "affected_features": ["f1"],
    "selection_criteria": "all",
    "percentage": 0.5,
    "mode": "new",
    "perturbate_data": {"distribution": "random"}
}

cluster_config = PuckTrick.make_remote_cluster_config(
    master_url="spark://XX.YY.ZZ.XY:7077/",
    num_executors=4,        # 4 workers (number of nodes in
                            # your SPARK cluster)
    executor_cores=4,       # Usa 4 core per worker
    executor_memory="15g",  # RAM memory
    driver_memory="4g"
)

OBJ = PuckTrick(dataframe=dataset,
                engine=Engine.SPARK,
                remote_cluster=cluster_config)

run_duplicated = OBJ.duplicated(train_df, strategy=strategy)
```

Capitolo 2

Sfide Tecniche

La conversione della libreria concepita per operare su strutture dati in-memory (Pandas) verso un framework distribuito (Apache Spark) ha posto una serie di sfide tecniche, derivanti dalle differenze architetturali profonde tra i due paradigmi. In questa sezione si analizzano i problemi le strategie risolutive adottate, incontrati nello sviluppo delle fasi di conversione dei metodi e di ristrutturazione della libreria menzionati nel capitolo precedente.

2.1 Assenza di indici posizionali in Spark

Una delle differenze più rilevanti tra Pandas e Spark risiede nel modello di accesso ai dati. Pandas espone un sistema di indicizzazione posizionale che, tramite i costrutti `iloc` e `loc`, consente di selezionare righe e colonne per posizione numerica o per etichetta. Questo paradigma è alla base di numerose operazioni nella versione originale di PuckTrick, in particolare per il campionamento di sottoinsiemi di righe da sporcare e per le operazioni di merge tra dataset originale e dataset modificato.

Apache Spark, per contro, opera su collezioni distribuite (RDD e DataFrame) prive di un ordinamento intrinseco e di un indice posizionale. Tale assenza è una conseguenza diretta dell'architettura distribuita: poiché i dati sono partizionati su nodi differenti del cluster, la nozione stessa di "riga i-esima" non è definita in modo univoco senza un'operazione di ordinamento globale, che risulterebbe computazionalmente onerosa.

La soluzione adottata consiste nell'utilizzo della funzione nativa:

```
monotonically_increasing_id()
```

Questa assegna a ciascuna riga un identificativo numerico univoco e monotonicamente crescente. Taleidentificativo, denominato `"_pucktrick_row_id"`, viene aggiunto al DataFrame al momento dell'inizializzazione della classe `PuckTrick` e funge da surrogato dell'indice posizionale, consentendo operazioni di join, filtraggio e campionamento deterministico senza ricorrere a odinamenti globali.

2.2 Lazy evaluation vs eager evaluation

Pandas adotta un modello di esecuzione *eager*: ogni istruzione viene eseguita immediatamente e il risultato è disponibile in memoria al termine della chiamata. Spark, al contrario, implementa un modello di esecuzione *lazy*: le operazioni di trasformazione (quali `filter`, `withColumn`, `select`) vengono registrate in un piano logico di esecuzione (*Directed Acyclic Graph*, DAG) ma non vengono effettivamente eseguite fino a quando non viene invocata un'azione (ad esempio `count`, `collect`, `show`). Questo meccanismo consente al Catalyst Optimizer di Spark di riordinare, fondere e ottimizzare le operazioni prima della loro esecuzione materiale.

Questa differenza ha richiesto una ristrutturazione del flusso logico dei moduli di iniezione errori. Nel codice Pandas originale, risultati intermedi vengono spesso ispezionati, confrontati o utilizzati come input per decisioni condizionali nel corso dell'esecuzione. Nella versione Spark, è stato necessario strutturare il codice in modo da costruire pipeline complete di trasformazioni e materializzarle esclusivamente al momento della restituzione del risultato finale. Ciò ha comportato un ripensamento della logica di controllo in diversi moduli (in particolare in `noisy` e `outliers`, dove la decisione sulla strategia di perturbazione dipende da proprietà del dato; ad esempio se una colonna è categorica o numerica) che richiedono un'azione per essere determinate

2.3 Campionamento DataFrame distribuiti

L'iniezione di errori in PuckTrick si basa fondamentalmente sul campionamento: data una percentuale specificata nella strategia, la libreria seleziona un sottoinsieme di righe su cui applicare la corruzione. In Pandas, questa operazione è deterministica (metodo `sample(n=...)`): restituisce esattamente n righe.

In Spark (metodo `sample(fraction=...)`) si opera su base probabilistica: ciascuna riga viene inclusa nel campione con una probabilità pari alla frazione specificata, il che produce un risultato approssimato (il cui numero di righe segue una distribuzione binomiale). Per dataset di piccole dimensioni, la deviazione rispetto al numero atteso può essere significativa in termini relativi, ma per dataset più grandi (usecase principale di spark) la variazione può diventare signigicativa.

Per garantire la coerenza funzionale con il backend Pandas (per tutti i suoi moduli), è stata implementata una strategia di campionamento personalizzata che prevede:

1. un sovracampionamento iniziale con una frazione maggiorata per assicurare un numero sufficiente di candidati,

2. l'applicazione di `limit(n)` per troncare il risultato al numero esatto di righe desiderato.

Questa tecnica introduce un overhead computazionale trascurabile ma garantisce che la percentuale di corruzione specificata nella strategia sia rispettata con precisione.

2.4 Gestione dei valori nulli

Pandas e Spark presentano differenze semantiche rilevanti nella rappresentazione dei valori mancanti. Per esempio: mentre Pandas distingue tra `None` e `NaN` (con comportamenti che variano a seconda del tipo della colonna) Spark, invece, utilizza uniformemente il valore `null` per rappresentarel'assenza di dato, indipendentemente dal tipo della colonna.

Questa discrepanza ha richiesto un'attenzione particolare nella conversione del modulo `missing`, dove l'obiettivo è proprio l'iniezione di valori nulli, e nel modulo `noisy`, dove la perturbazione di valori numerici deve distinguere correttamente tra celle contenenti dati validi e celle già nulle.

La soluzione adottata prevede l'utilizzo sistematico di:

1. cast espliciti (`cast("string")`, `cast("double")`)
2. controlli dedicati (`isNull()`, `isNotNull()`)

Questo per garantire un trattamento coerente dei valori nulli in ogni modulo del backend Spark.

2.5 Gestione dei path

La riorganizzazione della struttura del repository ha introdotto problemi dirottura sistematica dei riferimenti relativi utilizzati nei test e negli script di supporto. I moduli di test, in particolare, contenevano path relativi codificati staticamente (ad esempio per il caricamento di dataset CSV) che risultavano invalidi nella nuova struttura.

La risoluzione ha richiesto un aggiornamento sistematico di tutti i riferimenti, adottando `pathlib.Path` con path relativi alla root del progetto anziché path assoluti o relativi alla posizione del singolo file. Questa scelta, oltre a risolvere il problema immediato, ha migliorato la portabilità del progetto tra sistemi operativi differenti.

2.6 Attendibilità dei test

Durante la fase di validazione, è emerso che i test eseguiti producevano risultati di perturbazione non deterministici con il backend Spark. La causa risiede nella natura probabilistica del campionamento distribuito discussa nella sezione 2.3: l'approssimazione introdotta da `sample()` può generare campioni vuoti o di dimensione significativamente diversa da quella attesa, causando il fallimento di asserzioni nei test.

La soluzione ha comportato due interventi complementari:

1. l'incremento delle dimensioni dei dataset utilizzati nei test, portandoli a un ordine di grandezza sufficiente a rendere trascurabile l'errore di campionamento;

2. l'adattamento delle soglie di tolleranza nelle asserzioni dei test, accettando una deviazione percentuale minima rispetto al valore atteso quando il backend è Spark. Questo compromesso riflette una proprietà intrinseca dei sistemi distribuiti, in cui la precisione assoluta viene scambiata con la scalabilità.

Capitolo 3

Spark Setup

Al fine di validare il backend Spark di PuckTrick in un contesto distribuito reale (e non limitarsi alla modalità locale utilizzata durante lo sviluppo) è stato configurato un cluster Apache Spark in modalità *Standalone* su un insieme di macchine virtuali messe a disposizione dall'Università. L'infrastruttura è composta da un nodo master e quattro nodi worker, con risorse hardware eterogenee: tre macchine virtuali leggere (VM2, VM3, VM4) dotate di 4 core e 16 GB di RAM ciascuna, e una macchina più potente (worker principale) con 16 core e 32 GB di RAM.

3.1 Configurazione accesso SSH tra i nodi

Il primo requisito per il funzionamento di un cluster Spark Standalone è la possibilità per il nodo master di comunicare con i worker via SSH **senza autenticazione interattiva**. Questa condizione è necessaria affinché il master possa avviare e arrestare i demoni Spark sui nodi remoti tramite gli script `start-all.sh` e `stop-all.sh`.

È stata generata sul master una coppia di chiavi RSA senza passphrase e la chiave pubblica è stata distribuita su ciascun worker tramite `ssh-copy-id`:

```
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa

ssh-copy-id user@worker1
ssh-copy-id user@worker2
ssh-copy-id user@worker3
ssh-copy-id user@worker4
```

Questa operazione, ha richiesto la preventiva verifica che il servizio SSH fosse attivo su tutte le macchine e che le regole del firewall consentissero le connessioni in ingresso sulla porta 22.

3.2 Registrazione dei worker e configurazione dell'ambiente

Una volta stabilita la connettività SSH, è stato necessario dichiarare al master l'elenco dei nodi worker. Ciò avviene tramite la creazione del file

`$SPARK_HOME/conf/workers`

Questo contenente gli hostname o gli indirizzi IP dei quattro worker, uno per riga. Questo file viene letto dagli script di orchestrazione di Spark per determinare su quali macchine avviare i processi worker:

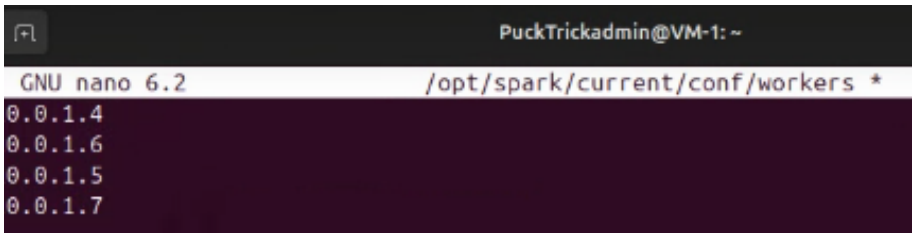

Figura 3.1: IP workers

Il passo successivo è stata la definizione del file `spark-env.sh`, che specifica le variabili d'ambiente critiche per il funzionamento del cluster. I parametri configurati sono stati:

```
export JAVA_HOME=/usr/lib/jvm/java-17

export SPARK_MASTER_HOST=10.0.1.8
export SPARK_MASTER_PORT=7077

export SPARK_WORKER_CORES=4
export SPARK_WORKER_MEMORY=15g
```

La scelta dei valori di `SPARK_WORKER_CORES` e `SPARK_WORKER_MEMORY` è stata guidata da un principio di allocazione aggressiva ma stabile: poiché le macchine erano dedicate esclusivamente a Spark, si è deciso di assegnare a ciascun worker la quasi totalità delle risorse disponibili, riservando circa 1 GB di RAM e un margine minimo di CPU per il sistema operativo e per il demone Spark stesso. Per il worker principale, dotato di risorse significativamente superiori, si è optato per una configurazione iniziale conservativa (4 core, 15 GB) allineata a quella delle macchine più piccole, con l'intento di evitare che un singolo nodo monopolizzasse l'assegnazione dei task a scapito del bilanciamento del carico.

3.2.1 Criticità di questa fase

Distribuzione dei file di configurazione. La procedura standard prevede la copia del file `spark-env.sh` dal master ai worker tramite `scp`. Tuttavia, nel contesto specifico delle macchine utilizzate, questa operazione falliva sistematicamente con errori di tipo *access denied*, presumibilmente a causa di permessi insufficienti sulla directory di destinazione o di restrizioni nelle policy di sicurezza delle VM.

La soluzione adottata è stata la creazione manuale del file su ciascun nodo, accedendo individualmente via SSH e replicando il contenuto del file di configurazione. Sebbene meno elegante rispetto alla distribuzione automatizzata, questo approccio ha garantito la corretta configurazione di tutti i nodi.

3.3 Creazione delle directory di logging

Spark richiede la presenza di una directory di lavoro:

`$SPARK_HOME/work`

con permessi di scrittura per l'utente che esegue i processi. L'assenza di tale directory (non creata automaticamente dall'installazione di Spark) causava il fallimento silenzioso dell'avvio dei worker. È stato necessario creare manualmente la directory su ogni nodo e assegnare i permessi corretti all'utente di servizio.

3.4 Apertura delle porte di rete.

Il funzionamento del cluster richiede l'apertura di diverse porte sul firewall di ciascuna macchina:

```
sudo ufw allow 8080/tcp
sudo ufw allow 7077/tcp
```

Le porte usate sono:

Porta	Servizio	Direzione
7077	Spark Master (comunicazione cluster)	Worker → Master
8080	Web UI del Master	Accesso esterno

Tabella 3.1: Porte utilizzate dal cluster Spark

La mancata apertura della porta 7077 impedisce ai worker di registrarsi presso il master, producendo un cluster apparentemente attivo ma privo di risorse computazionali. La Web UI sulla porta 8080 si è rivelata uno strumento diagnostico indispensabile per verificare lo stato del cluster, il numero di worker connessi e l'allocazione delle risorse.

3.5 Verifica del funzionamento

La corretta attivazione del cluster è stata verificata in prima istanza tramite una richiesta HTTP al servizio Web UI del master:

```
curl -s http://localhost:8080/ | grep -i "spark master"
```

La risposta positiva ha confermato l'avvio del processo master.

Successivamente, l'accesso alla Web UI ha permesso di verificare la registrazione di tutti e quattro i worker e la corretta allocazione delle risorse (core e memoria) dichiarate nella configurazione:

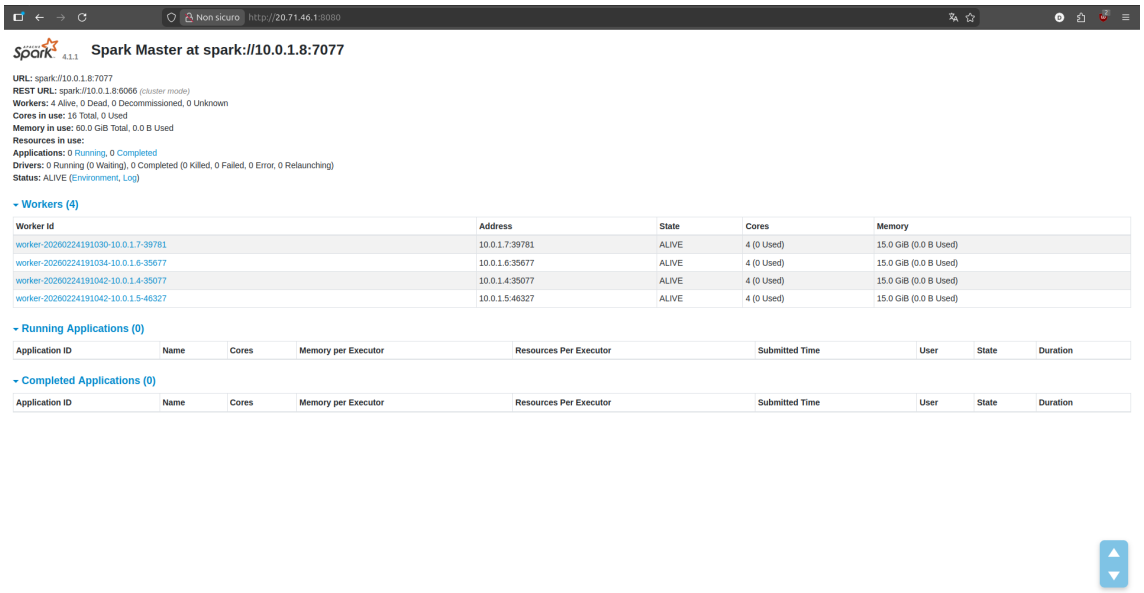

Figura 3.2: Dashboard Spark

Capitolo 4

Performance analisys

4.1 Analisi dei risultati di performance: confronto Pandas vs Spark

Una volta finito il setup remoto della libreria siamo potuti procedere con l'analisi delle performance di libreria.

4.1.1 Protocollo sperimentale

Il benchmark di confronto è stato eseguito su un dataset sintetico (`fake_dataset.csv`) contenente **1.000.000 di righe** con colonne di diversa tipologia:

- f1: colonna di tip float
- f2: colonna di tipo int (insieme numeri interi)
- f3: colonna categorica (string) con 3 classi
- f4: colonna categorica (string) con 3 classi
- f5: colonna categorica (int) con 6 classi
- date: colona di tipo data
- target: colonna di tipo binaria

Questa scelta progettuale è fondamentale per l'interpretazione dei risultati, poiché il tipo di dato della colonna influenza in modo determinante le performance dei backend Pandas e Spark

Ciascun modulo di iniezione errori è stato eseguito su ogni colonna singolarmente, applicando la medesima strategia per ambi i backend:

```
{
  "affected_features": ["<colonna>"],
  "mode": "new",
  "percentage": 0.5,
  "perturbate_data": {
    "distribution": "random"
  },
  "selection_criteria": "all"
}
```

L'esecuzione è stata sequenziale (con pulizia della memoria tra un esecuzione e l'altra). I tempi sono stati misurati e salvati in formato CSV [1], del quale grafichiamo i risultati di seguito 4.1.

4.1.2 Disclaimer:

Come possibile vedere dall'immagine a pagina seguente, il metodo `labels` è stato fatto runnare solo sul seguente subset di colonne:

- f3: colonna categorica (string) con 3 classi
- f4: colonna categorica (string) con 3 classi
- f5: colonna categorica (int) con 6 classi
- target: colonna di tipo binaria

Questo scelta è causata del fatto che questo metodo introduce rumore di etichetta nelle colonne selezionata sipportando però solo target binari o categorici.

4.1.3 Risultati del Paragone tra Backends

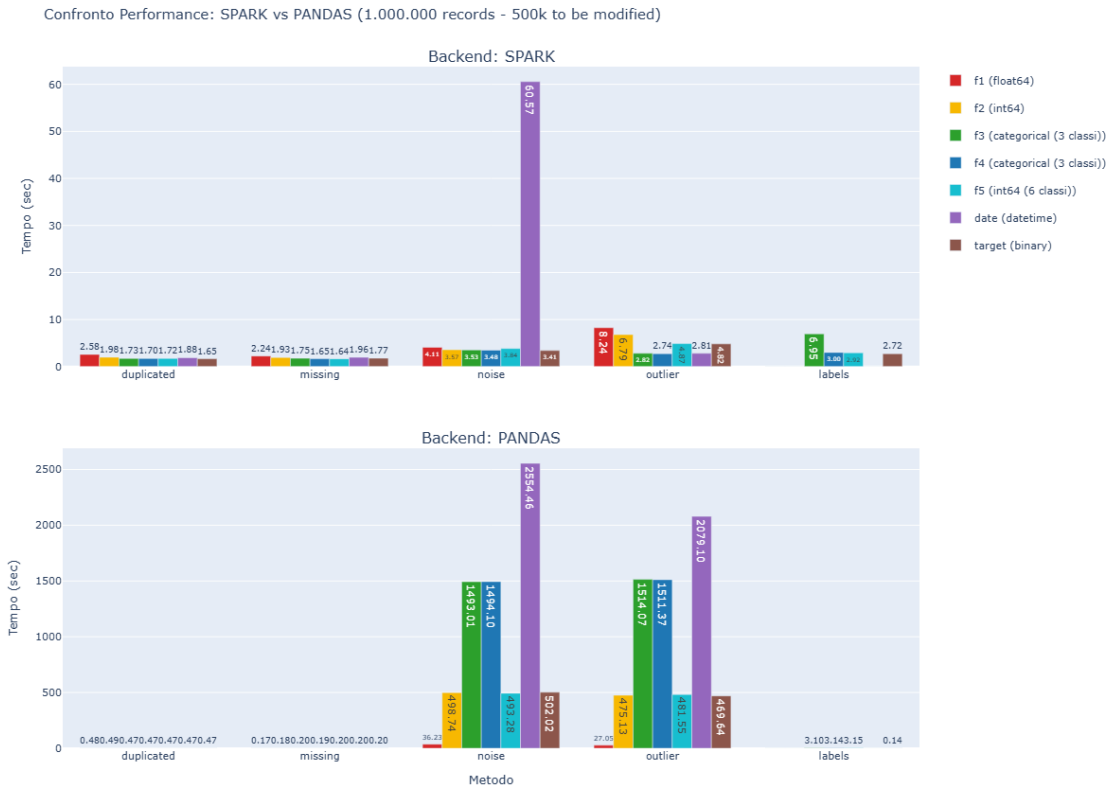

Figura 4.1: Risultati raggrupati per backend e per metodo

Osservando l'immagine precedente 4.1 risulta evidente ci siano 2 gruppi:

- metodi di libreria la quale implementazione Pandas risulta sempre più veloce rispetto a Spark (comprende i metodi `duplicated` e `missing`)
- metodi di libreria che presentano grosse differenze tra tempi di computazione dei due backend (comprende i metodi `noise`, `outlier` e `labels`)

Metodi complessità computazionale bassa: `duplicated` e `missing`

A primo impatto può semprare strano che, pur avendo un dataset da un milione di righe, ci possano essere dei metodi che seppur usando il backend Pandas risultino essere più veloci rispetto a Spark.

In secondo luogo però dopo una riflessione più attenta questo risultato è perfettamente deducibile: i moduli `duplicated` e `missing` sono implementati da delle operazione di complessità computazionale estremamente bassa! Di conseguenza si avrà che la spiegazione di questo divario di tempi computazionali tra Pandas (circa 0.5 secondi) e Spark (circa 2 secondi) risiede nell'**overhead strutturale** del framework Spark. Abbiamo infatti che per il modulo Spark viene sempre eseguita la compilazione del piano di esecuzione (DAG), la serializzazione dei dati tra il driver Python e la JVM, la gestione delle partizioni e

il coordinamento dell'esecuzione distribuita. Tutti questi fattori introducono un costo fisso stimabile in circa 1.2–1.5 secondi che, per operazioni intrinsecamente semplici su dataset che non eccedono la capacità della memoria di una singola macchina, non viene ammortizzato da alcun guadagno nella parallelizzazione.

È significativo inoltre notare come la prima esecuzione di Spark `duplicated` (`f1`, 2.58s) sia sensibilmente più lenta delle successive (1.65–1.98s), un fenomeno attribuibile al "warm-up" della JVM e alla compilazione JIT (Just-In-Time) del bytecode durante la prima invocazione. Questo fenomeno è infatti osservabile per tutti i metodi Spark che operano sulla prima colonna (f1). Togliendo infatti circa 1.5 dai tempi di computazione di ciasuna elaborazione fatta su questa colonna si hanno tempo di esecuzione pressapoco allineati per tutti i metodi Spark, nonostrante ci sono delle eccezzioni a questa regola.

Metodi complessità computazionale alta: `noise`, `outlier` e `labels`

Nota: Prima di discutere i risultati compiuti da questi metodi si fa una piccola nota sul metodo `labels`; potrebbe infatti sembrare che questo metodo ricada nella casistica precedente, avendo che tutti i risultati ottenuti da backend Pandas abbiano tempi computazine molto inferiori rispetto che tempi con Spark. Questo però è devuto al fatto che per la nostra casistica abbiamo che tutte le colonne fornite al metodo `labels` hanno pochiessime classi al loro interno:

- f3: 3 classi
- f4: 3 classi
- f5: 6 classi
- target: 2 classi

Questo porta il metodo ad avere tempi di esecuzione estremamente bassi. Si ha infatti `labels` (Nella sua implementazione Pandas) ha complessità direttamente proporzionale al numero di classi inserito. Provando infatti ad aumentare di molto il numero di classi si inzierebbero a notare tempi di computazione estremamente più alti; che superano di gran lunga i tempi di Spark. Essendo dunque la complessità di questo metodo molto alta quando proporzionata al numero di classi specificate si considera il metodo `labels` come appartenente a questa sezione.

Se osserviamo dunque i tempi di esecuzione, per i metodi con complessità computazionale di calcolo maggiore, possiamo trovare una netta differenza tra il tempo impiegato da Pandas e quello impiegato da Spark. Emerge infatti una disparità di costo per tipo di dato e metodo usato nel backend Pandas. Abbiamo che questa disparità può essere sintetizzata come:

$$\text{float} < \text{int} < \text{string} < \text{date}$$

I tempi Spark, al contrario, sono **straordinariamente uniformi** attraverso tutti i tipi di dato con valori compresi tra 2 e 8 secondi, indipendentemente dal tipo della colonna. Questa uniformità (e ampio miglioramento rispetto le performance di Pandas) è una conseguenza diretta del paradigma colonnare di Spark: tutte le operazioni (che si tratti di perturbazione di float, sostituzione di stringhe o manipolazione di interi categorici) vengono compilate dal Catalyst Optimizer in piani di esecuzione nativi JVM; ed eseguite sulle partizioni in parallelo, senza mai transitare per l'interprete Python e senza subire il costo differenziale dei tipi di dato (che penalizza Pandas). Abbiamo infatti che la gerarchia ti tempi proporzionali ai tipi ossrevata nel Backend Pandas scompare completamente in Spark. Rimane però un unica eccezione per la colonna `date` (60.6 secondi) nell'esecuzione della funzione `Noise`, questa risalta per le sue tempistiche di esecuzione particolarmente alte rispetto tutte le altre istanze Spark.

La ragione fondamentale di questa disparità risiede nella definizione stessa della funzione nel backend Spark. Questa funzione esprime infatti la stessa logica applicata dal backend Pandas (perturbazione tramite cicli Python che iterano su ciascuna riga selezionata, applicando trasformazioni individuali) ma tramite trasformazioni colonnari native (`withColumn`, `when`, `rand()`) che vengono compilate in codice JVM ottimizzato ed eseguite senza mai transitare per l'interprete Python. Avendo, però, creato una riproduzione fedele del codice Pandas nel backend Spark ha portato alla stesura di codice non ottimizzato per la gestione delle colonne di tipo data. Possiamo, infatti, osservare che anche per il backend Pandas il tipo data porta ad avere sempre elaborazioni più lunghe usando il metodo `Noise`.

Abbiamo dunque che i risultati del benchmark delle righe delineano un quadro chiaro e coerente: A 1 milione di righe, Spark è già vantaggioso per `noise`, `outlier` e `labels` su tutti i tipi di colonna; e lo è in modo schiacciante sulle colonne stringa e date. Mentre abbiamo che per `duplicated` e `missing` il punto di pareggio si colloca a volumi di dati superiori, dove l'overhead fisso di Spark diventa trascurabile rispetto al tempo di elaborazione.

4.2 Spark stress test

I risultati dello stress test con metodo `duplicated` su cluster remoto Spark [**1**] mostrano un comportamento di scalabilità sostanzialmente lineare: partendo da circa 3,6 secondi per 1 milione di righe, il tempo per singola iterazione raddoppia progressivamente man mano che il dataset raddoppia, arrivando a ∼84,6 secondi per 128 milioni di righe

È interessante osservare che le prime tre iterazioni (1M, 2M e 4M di righe) presentano tempi molto simili tra loro (circa 3/3,5 sec), suggerendo che a bassa cardinalità il costo dominante è la fase di inizializzazione del job Spark piuttosto che il volume dati effettivo .

A partire dagli 8 milioni di righe il comportamento diventa sempre più proporzionale al dato, con la curva che scala in modo quasi perfettamente lineare, visibile chiaramente nel grafico sotto riportato [Figura 4.2], senza segnali di colli di bottiglia anomali, collisioni di shuffle o degrado delle risorse del cluster fino a 128M di righe.

La strategia adottata [**strategia 4.2.1**], riportata è stata applicata coerentemente in tutte le 8 iterazioni, confermando che la linearità osservata è intrinseca al metodo `duplicated` e non a variazioni nei parametri di test.

In sintesi, il metodo si dimostra scalabile e prevedibile su cluster Spark remoto, con un tempo totale di circa 173,6 secondi per l'intera suite da 128M di righe, il che lo rende adatto a dataset di grandi dimensioni in produzione, a patto di considerare il fisso overhead iniziale per job di dimensioni ridotte.

4.2.1 Strategy di Test Utilizzata

La configurazione della strategy è codificata in JSON e passata al framework PuckTrick:

```
{
  "affected_features": ["f1"],
  "mode": "new",
  "percentage": 0.5,
  "perturbate_data": {
    "distribution": "random"
  },
  "selection_criteria": "all"
}
```

Vengono riportati adesso i dati graficati:

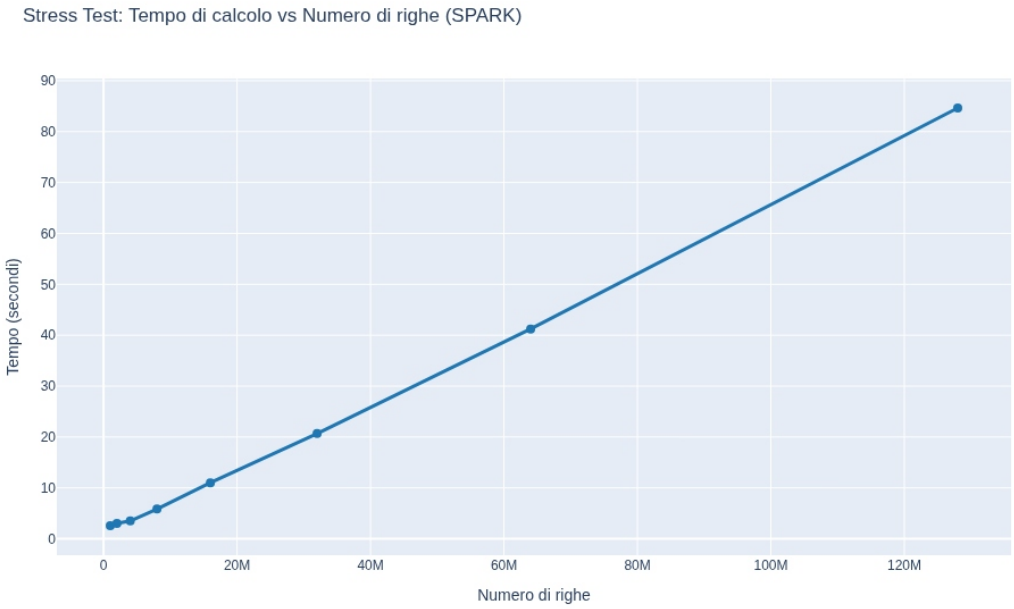

Figura 4.2: Comportamento in termini di tempo di Spark allo scalare della grandezza del dataset

Conclusioni

In questo lavoro abbiamo dunque confermato che Pandas si conferma estremamente efficiente per operazioni semplici come la gestione dei duplicati o dei valori mancanti (duplicated, missing) su dataset inferiori a 4 milioni di righe, con tempi di esecuzione al di sotto dei secondi grazie alla vettorizzazione NumPy. Tuttavia, le sue prestazioni degradano sensibilmente su task più complessi (noise, outlier, labels), in particolare con stringhe e date, dove l'approccio row-by-row può portare i tempi a diverse ore anche su dataset di 1 milione di righe.

Spark, al contrario, introduce un overhead iniziale fisso di circa 3 secondi, scala inoltre in modo lineare con la dimensione del dataset. Elimina infatti le differenze di costo computazionale legate al tipo di dato e offre prestazioni nettamente superiori nelle operazioni più onerose, risultando la scelta ideale per ambienti di produzione e casi d'uso su big data rispetto il backend Pandas, confermando quindi il caso d'uso nativo del backend Spark.

La conversione di PuckTrick a Spark dunque ha rappresentato un successo: tutti i metodi sono stati implementati con una chiara suddivisione progettuale. Inoltre il punto di accesso unico tramite la classe PuckTrick si è rivelato una scelta vincente, poiché semplifica notevolmente l'utilizzo della libreria e riduce la complessità percepita dall'utente finale.

Restano comunque alcune aree di miglioramento. È necessario un refactoring del modulo Pandas per rimuovere il codice non più utilizzato e ottimizzare la logica di esecuzione dei metodi. Inoltre, occorre ridurre il debito tecnico sulla documentazione: attualmente essa copre solo i singoli metodi, ma l'aggiunta di flow chart dei flussi di lavoro renderebbe il codice più chiaro e favorirebbe l'allineamento tra gli sviluppatori. Inoltre a livello di codice, per grarantire una user experience migliore, sarebbe poi possibile implementare la conversione automatica del dataset fornito, al formato supportato dal backend, al momento dell'istanza della libreria pucktrick.

Dai test è poi emerso anche una mancata ottimizzazione nel metodo outliers, che presenta tempi di computazione molto alti nella gestione delle date sia nella versione Pandas sia in quella Spark (Pur mantenendo il paradigma che Spark rimane molto più efficiente).

In conclusione, i risultati ottenuti sono pienamente in linea con le aspettative iniziali, e confermano la solidità dell'approccio Spark come evoluzione naturale della libreria.

Bibliografia

[1] Daniel Satriano e Francesco Cavallini. *Risultati Stress Test Spark con Metodo Duplicated su Cluster Remoto*. `https://drive.google.com/drive/folders/1f_ZjlX5Oi_-FMz99r3kNOnE_nadP-M1j`. Ver. 1.0. Dataset CSV: tempi di esecuzione iterazioni da 1M a 128M righe, metodo duplicated, backend SPARK. Feb. 2026 (cit. alle pp. 14, 18).